Desenvolvimento Low Code

Projeto Sistema Ominichanel

Emmanuel Duarte de Oliveira

Universidade de Cuiabá (UNIC)

Low Code; Sistema Ominichanel

oliveiraemmanuel658@gmail.com

1. Introdução

O desenvolvimento de software para o setor de saúde enfrenta um desafio de modernização. As metodologias tradicionais de comunicação em clínicas, muitas vezes segmentadas em canais isolados como telefone, e-mail e múltiplos aplicativos de mensagens, criam uma experiência de usuário fragmentada e uma gestão operacional ineficiente. No cenário contemporâneo, a expectativa de pacientes e gestores mudou de um atendimento reativo para uma demanda por plataformas *omnichannel* integradas, que centralizam WhatsApp, Instagram, Facebook, e-mail e chat em tempo real em uma única interface.

Essa transformação é impulsionada por stacks de tecnologia modernas, como o React, para a criação de interfaces de usuário ricas, e backends como o Supabase, que oferecem bancos de dados e WebSockets para funcionalidades em tempo real. Contudo, a implementação de tal sistema a partir do zero apresenta barreiras significativas. A complexidade de desenvolver uma arquitetura segura que gerencie múltiplos níveis de acesso (como pacientes, atendentes e gerentes), que mantenha a persistência de históricos de chat e forneça dashboards analíticos, é um processo tradicionalmente caro e demorado, exigindo equipes de desenvolvimento altamente especializadas.

Este artigo foca na análise de uma nova abordagem para solucionar esse conflito: o **desenvolvimento low-code assistido por Inteligência Artificial (IA)**. Assim como frameworks como o Next.js resolveram problemas de performance do React , plataformas de desenvolvimento assistido por IA, como o Lovable.dev, estão mudando o paradigma da criação de aplicações complexas. Em vez da codificação manual, essa metodologia utiliza engenharia de prompt para gerar código-fonte otimizado (incluindo TypeScript, Tailwind CSS e Shadcn-ui).

2. Objetivo

A seção de Introdução estabeleceu o desafio central no desenvolvimento de software para saúde: a tensão entre a necessidade urgente de sistemas de atendimento *omnichannel* integrados e a alta complexidade técnica, de custo e de tempo associada ao desenvolvimento tradicional. Este artigo analisa a aplicação de uma metodologia de desenvolvimento low-code assistida por Inteligência Artificial como uma intervenção arquitetônica deliberada para resolver esse conflito.

Com base nessa problemática, o **objetivo geral** deste estudo é analisar e documentar o processo de desenvolvimento de um sistema *omnichannel* full-stack para clínicas, o

`MediChannel`, utilizando uma metodologia centrada em plataformas de IA (Lovable.dev e Gemini).

Para alcançar este objetivo principal, foram definidos os seguintes **objetivos específicos**:

1. **Contextualizar** a relevância da abordagem de desenvolvimento assistido por IA em contraste com o desenvolvimento manual tradicional, destacando as vantagens na automação da criação de arquiteturas complexas.
2. **Diagnosticar e detalhar** os requisitos técnicos essenciais de uma aplicação clínica moderna, focando na integração de múltiplos canais (WhatsApp, Instagram, etc.), na segurança de acesso por nível de usuário (Paciente, Atendente, Gerente) e na funcionalidade de chat em tempo real.
3. **Analisar tecnicamente** a implementação da plataforma Lovable.dev na geração do `MediChannel`, examinando como a IA gerou a stack tecnológica (React, Vite, TypeScript, Supabase) e abordou diretamente os desafios de segurança (autenticação JWT, Row Level Security) e comunicação em tempo real (WebSocket).
4. **Sintetizar as descobertas**, demonstrando o impacto da metodologia na entrega de um artefato de software completo e funcional, e solidificando o argumento de que o desenvolvimento assistido por IA é um método viável para a produção de aplicações empresariais complexas.

3. Metodologia

Para atender aos objetivos propostos, a metodologia adotada foi um **estudo de caso técnico-analítico** focado no processo de desenvolvimento do sistema de atendimento *omnichannel* `MediChannel`, utilizando uma plataforma de desenvolvimento assistido por Inteligência Artificial.

Diferente das abordagens de desenvolvimento tradicionais, que envolvem a codificação manual linha por linha em um IDE , esta pesquisa utilizou uma metodologia de **engenharia de prompt**. O processo foi centrado na plataforma **Lovable.dev**, um construtor de aplicações full-stack que utiliza IA para interpretar prompts de linguagem natural e gerar código-fonte otimizado para produção. O **Gemini** foi utilizado em conjunto para auxiliar no refinamento da lógica de negócio e na geração de código específico.

O desenvolvimento do projeto foi recontextualizado metodologicamente da seguinte forma:

1. **Fase de Geração e Iteração (90% do Desenvolvimento):** Esta fase principal consistiu na utilização intensiva de prompts de linguagem natural dentro do Lovable para gerar a arquitetura e os componentes da aplicação. As iterações foram feitas descrevendo as funcionalidades desejadas (ex: "Crie um sistema de autenticação com 3 níveis de usuário", "Implemente um chat em tempo real com Supabase", "Crie um dashboard gerencial com gráficos de performance"). A IA da plataforma gerou o código correspondente na stack tecnológica moderna:

- **Frontend:** React (Vite), TypeScript, Tailwind CSS e Shadcn-ui.

- **Backend:** Integração com Supabase para autenticação, banco de dados (PostgreSQL com Row Level Security) e chat em tempo real (WebSocket).
- **Lógica de Negócio:** A IA, auxiliada pelo Gemini, gerou a camada de serviços (`AuthService`, `ChatService`) e hooks customizados (`useChat`) para modularizar e gerenciar o estado da aplicação.
- **Fase de Ajustes Finais e Versionamento (10% do Desenvolvimento):** Esta etapa final da metodologia focou no refinamento do produto gerado e na sua extração para um fluxo de trabalho profissional. Isso envolveu o uso das integrações nativas do Lovable, especificamente a **conexão com o GitHub**. O código-fonte final, gerado e otimizado pela IA , foi comitado em um repositório Git, permitindo o versionamento. Por fim, a aplicação foi implantada (deploy) na própria infraestrutura do Lovable, tornando-a publicamente acessível para validação.

Aqui está a proposta para a seção de **Resultados**, que descreve objetivamente os produtos e artefatos gerados pela sua metodologia.

---

4. Resultados

A aplicação da metodologia de desenvolvimento assistido por IA, centrada na plataforma Lovable.dev e auxiliada pelo Gemini, produziu um conjunto de resultados tangíveis que atendem diretamente aos objetivos da pesquisa. Os resultados deste estudo de caso técnico são os seguintes:

1. Geração de um Artefato de Código Full-Stack (Stack Moderna) O resultado primário da metodologia de engenharia de prompt foi a geração bem-sucedida de um sistema *omnichannel* completo, o `MediChannel`. A plataforma, em resposta aos prompts, gerou uma aplicação moderna na stack tecnológica solicitada:

- **Frontend:** Uma aplicação em **React 18 com TypeScript** e **Vite**, garantindo performance no desenvolvimento e *type safety*.
- **Design System:** Utilização de **Tailwind CSS** e componentes **Shadcn-ui**, resultando em uma interface de usuário (UI) responsiva, consistente e com funcionalidades modernas como modo escuro/claro.
- **Backend (BaaS):** Integração completa com **Supabase** para autenticação (JWT) e gerenciamento de banco de dados PostgreSQL.

2. Implementação de Funcionalidades Complexas em Tempo Real Diferente de uma aplicação web estática, o resultado obtido é um sistema altamente dinâmico e complexo:

- **Autenticação e Segurança:** O sistema gerou um fluxo de autenticação seguro com registro, login e **três tipos de usuário** (Paciente, Atendente, Gerente). A segurança dos dados foi implementada via **Row Level Security** (RLS) no Supabase.
- **Chat em Tempo Real:** O resultado mais significativo foi um sistema de **chat em tempo real via WebSocket** (usando as *real-time subscriptions* do Supabase), com histórico de conversas persistente, notificações e uma fila unificada de atendimento.

- **Dashboards Segmentados:** Foram gerados três painéis de controle distintos e funcionais, cada um com seus próprios componentes e lógica de negócio: o **Painel do Paciente** (foco em agendamentos), o **Painel do Atendente** (foco em respostas rápidas e fila) e o **Painel do Gerente** (foco em métricas e relatórios).

3. Geração de Arquitetura de Software e Versionamento A fase de "ajustes finais" (10% do processo) produziu resultados metodológicos cruciais. O código-fonte gerado pela IA não foi monolítico; ele resultou em uma arquitetura de software profissional com clara separação de responsabilidades, incluindo:

- Uma camada de serviços (`AuthService`, `ChatService`, `DashboardService`).
- Hooks customizados do React (`useChat`, `useToast`) para encapsular lógicas reutilizáveis.
- O resultado mais significativo desta fase foi a integração bem-sucedida com o **GitHub**, onde o código-fonte completo foi comitado, estabelecendo um fluxo de trabalho de desenvolvimento (Git workflow) padrão. A aplicação foi então implantada (deploy) com sucesso, tornando-se funcional e acessível através do link da plataforma Lovable.

5. Conclusão

Este artigo teve como objetivo geral analisar a geração e otimização de um sistema de atendimento *omnichannel* para clínicas, o `MediChannel`, focando em como a metodologia de desenvolvimento assistido por IA e a arquitetura final (React/Supabase) resolvem os desafios centrais de complexidade, segurança e funcionalidade em tempo real.

A principal conclusão deste estudo é que as metodologias de desenvolvimento assistido por Inteligência Artificial, especificamente através da plataforma **Lovable.dev** e com o auxílio do **Gemini**, representam uma mudança paradigmática na forma como aplicações empresariais complexas são criadas. A pesquisa demonstrou que a engenharia de prompt evoluiu de uma ferramenta de prototipagem para um método de produção viável, capaz de gerar código-fonte de alta complexidade (full-stack, real-time) e pronto para produção.

Atingindo os objetivos específicos, este estudo de caso confirmou que:

1. A abordagem de desenvolvimento assistido por IA supera as limitações do desenvolvimento manual tradicional, pois permite a **implementação de arquiteturas complexas** (React, Supabase, WebSocket) em uma fração do tempo e custo habituais.
2. A metodologia resolveu o conflito central entre funcionalidade e segurança. Ao gerar uma aplicação com **autenticação segura e três níveis de usuário** (Paciente, Atendente, Gerente), além de **Row Level Security** no banco de dados, os desafios de segmentação de dados e permissões foram resolvidos nativamente pela IA.
3. Simultaneamente, o uso das *real-time subscriptions* do Supabase garantiu que a interatividade dinâmica de um **chat *omnichannel* em tempo real** fosse totalmente preservada e funcional, entregando a funcionalidade mais crítica do projeto.

Por fim, conclui-se que a fase de "ajustes finais", correspondente aos 10% do esforço, foi estrategicamente decisiva. A **integração bem-sucedida do projeto gerado pela IA com um repositório GitHub** provou ser o elo essencial, validando o código como um ativo de desenvolvimento padrão. O processo moveu o projeto de uma plataforma de geração de IA para um fluxo de trabalho de desenvolvimento profissional (Git workflow), pronto para deploy e acessível publicamente.

Em suma, este trabalho não apenas valida a arquitetura React/Supabase como a solução técnica ideal para aplicações *omnichannel* modernas, mas, mais importante, demonstra que o desenvolvimento assistido por IA é um método de produção pragmático, rápido e eficiente para alcançar essa arquitetura otimizada.

6. Referências

GOOGLE AI. Gerar saídas de texto com a API Gemini. Disponível em: https://ai.google.dev/gemini-api/docs/text-generation?hl=pt-br

GOOGLE DEVELOPERS. Escrever e ajustar código com o Gemini Code Assist. Disponível em: https://developers.google.com/gemini-code-assist/docs/write-code-gemini?hl=pt-br

GITHUB. Repositório [Nome do seu Repositório, ex: MediChannel]. Disponível em: https://docs.github.com/pt/repositories/creating-and-managing-repositories/quickstart-for-repositories

LOVABLE.DEV. Build something Lovable. Disponível em: https://lovable.dev/

LOVABLE.DEV. How to develop an app with AI. Disponível em: https://lovable.dev/blog/how-to-develop-an-app-with-ai

MDN WEB DOCS. O que é JavaScript?. Disponível em: https://developer.mozilla.org/pt-BR/docs/Learn_web_development/Core/Scripting/What_is_JavaScript

MEDICHANNEL. Aplicação MediChannel (Preview). 2025. Disponível em: https://preview--health-chat-unity.lovable.app/. Acesso em: 14 nov. 2025.

REACT. React: A JavaScript library for building user interfaces. Disponível em: https://react.dev/. Acesso em: 14 nov. 2025.

SHADCN-UI. Beautifully designed components built with Radix UI and Tailwind CSS. Disponível em: https://ui.shadcn.com/. Acesso em: 14 nov. 2025.

SUPABASE. Supabase Documentation: Build in a weekend, scale to millions. Disponível em: https://supabase.com/docs. Acesso em: 14 nov. 2025.

Desenvolvimento Low Code

TAILWIND CSS. Rapidly build modern websites without ever leaving your HTML. Disponível em: https://tailwindcss.com/. Acesso em: 14 nov. 2025.

VITE. Next Generation Frontend Tooling. Disponível em: https://vitejs.dev/. Acesso em: 14 nov. 2025.